

Home Loan Prediction

By-Varun Ladha(17CE049) 

In [2]:

```
import numpy as np
import pandas as pd
import seaborn as sns
```

In [36]:

```
import matplotlib.pyplot as plt
%matplotlib inline
```

In [37]:

```
train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")
```

In [38]:

```
train.head(5)
```

Out[38]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	LP001002	Male	No	0	Graduate	No	5849	
1	LP001003	Male	Yes	1	Graduate	No	4583	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	
4	LP001008	Male	No	0	Graduate	No	6000	

In [39]:

```
train.shape
```

Out[39]:

(614, 13)

In [123]:

```
test.head(5)
```

Out[123]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapr
0	LP001015	Male	Yes	0	Graduate	No	5720	
1	LP001022	Male	Yes	1	Graduate	No	3076	
2	LP001031	Male	Yes	2	Graduate	No	5000	
3	LP001035	Male	Yes	2	Graduate	No	2340	
4	LP001051	Male	No	0	Not Graduate	No	3276	

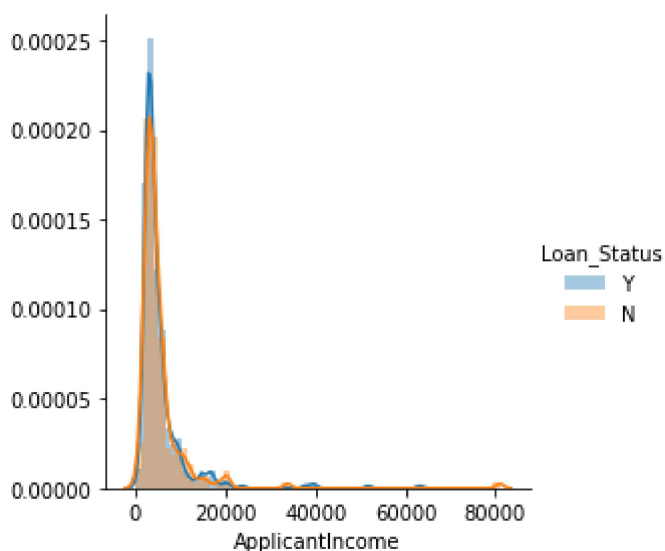
In []:

In [40]:

```
sns.set_style = ("whitegrid");
sns.FacetGrid(train,hue="Loan_Status",size= 4)\
    .map(sns.distplot,"ApplicantIncome")\
    .add_legend();
plt.show()
#seeing graph it can not be segrgated on basis of Applicant income
```

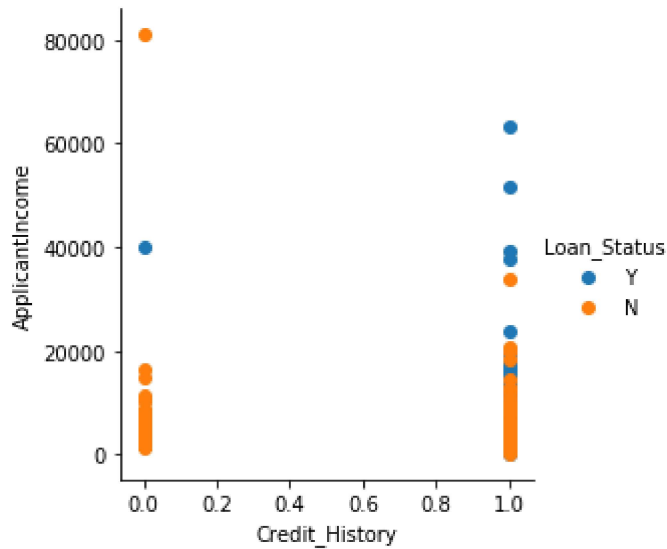
C:\Users\Varun Ladha\Anaconda3\lib\site-packages\seaborn\axisgrid.py:230: UserWarning: The `size` paramter has been renamed to `height`; please update your code.

```
warnings.warn(msg, UserWarning)
```



In [41]:

```
sns.set_style = ("whitegrid");
sns.FacetGrid(train,hue="Loan_Status",size= 4)\
    .map(plt.scatter,"Credit_History","ApplicantIncome")\
    .add_legend();
plt.show()
#so people with less than 20,000 salary and ) credit history are always no.
```



In [42]:

```
#Data Cleaning
train.isnull().sum()
```

Out[42]:

```
Loan_ID          0
Gender           13
Married          3
Dependents       15
Education        0
Self_Employed    32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term 14
Credit_History   50
Property_Area    0
Loan_Status      0
dtype: int64
```

In [103]:

```
train["Gender"].fillna(train["Gender"].mode()[0],inplace=True)
train["Married"].fillna(train["Married"].mode()[0],inplace=True)
train["Dependents"].fillna(train["Dependents"].mode()[0],inplace=True)
train["Self_Employed"].fillna(train["Self_Employed"].mode()[0],inplace=True)
train["Credit_History"].fillna(train["Credit_History"].mode()[0],inplace=True)
train['LoanAmount'].fillna(train['LoanAmount'].median(),inplace=True)
train['Loan_Amount_Term'].fillna(train['Loan_Amount_Term'].median(),inplace=True)

test["Gender"].fillna(test["Gender"].mode()[0],inplace=True)
test["Married"].fillna(test["Married"].mode()[0],inplace=True)
test["Dependents"].fillna(test["Dependents"].mode()[0],inplace=True)
test["Self_Employed"].fillna(test["Self_Employed"].mode()[0],inplace=True)
test["Credit_History"].fillna(test["Credit_History"].mode()[0],inplace=True)
test['LoanAmount'].fillna(test['LoanAmount'].median(),inplace=True)
test['Loan_Amount_Term'].fillna(test['Loan_Amount_Term'].median(),inplace=True)
```

In [44]:

```
train.isnull().sum()
```

Out[44]:

Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Loan_Status	0
dtype: int64	

In [45]:

```
#See the unique data
train.apply(lambda x: len(x.unique()))
```

Out[45]:

```
Loan_ID          614
Gender           2
Married          2
Dependents       4
Education        2
Self_Employed    2
ApplicantIncome  505
CoapplicantIncome 287
LoanAmount       203
Loan_Amount_Term 10
Credit_History   2
Property_Area     3
Loan_Status       2
dtype: int64
```

In [104]:

```
train1=train.drop("Loan_ID",axis=1)
test1= test.drop("Loan_ID",axis=1)
```

In [107]:

```
X= train1.drop('Loan_Status',1)
X_test = test1
X
```

Out[107]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAm
0	Male	No	0	Graduate	No	5849	0.0	
1	Male	Yes	1	Graduate	No	4583	1508.0	
2	Male	Yes	0	Graduate	Yes	3000	0.0	
3	Male	Yes	0	Not Graduate	No	2583	2358.0	
4	Male	No	0	Graduate	No	6000	0.0	
5	Male	Yes	2	Graduate	Yes	5417	4196.0	
6	Male	Yes	0	Not Graduate	No	2333	1516.0	
7	Male	Yes	3+	Graduate	No	3036	2504.0	

In [90]:

```
y=train1["Loan_Status"].values
```

In [91]:

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
```

In [109]:

```
X['Gender']=encoder.fit_transform(X['Gender'])
X['Married']=encoder.fit_transform(X['Married'])
X['Education']=encoder.fit_transform(X['Education'])
X['Self_Employed']=encoder.fit_transform(X['Self_Employed'])
X['Dependents']=encoder.fit_transform(X['Dependents'])
X['Property_Area']=encoder.fit_transform(X['Property_Area'])

X_test['Gender']=encoder.fit_transform(X_test['Gender'])
X_test['Married']=encoder.fit_transform(X_test['Married'])
X_test['Education']=encoder.fit_transform(X_test['Education'])
X_test['Self_Employed']=encoder.fit_transform(X_test['Self_Employed'])
X_test['Dependents']=encoder.fit_transform(X_test['Dependents'])
X_test['Property_Area']=encoder.fit_transform(X_test['Property_Area'])
```

In [110]:

```
X.head(5)
```

Out[110]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	1	0	0	0	0	5849	0
1	1	1	1	0	0	4583	1508
2	1	1	0	0	1	3000	0
3	1	1	0	1	0	2583	2358
4	1	0	0	0	0	6000	0

In [111]:

```
X_test.head(5)
```

Out[111]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	1	1	0	0	0	5720	
1	1	1	1	0	0	3076	150
2	1	1	2	0	0	5000	180
3	1	1	2	0	0	2340	254
4	1	0	0	1	0	3276	

```
train1.corr()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
ApplicantIncome	1.000000	-0.116605	0.565181	-0.046531	0.000000
CoapplicantIncome	-0.116605	1.000000	0.189218	-0.059383	0.000000
LoanAmount	0.565181	0.189218	1.000000	0.036960	0.000000
Loan_Amount_Term	-0.046531	-0.059383	0.036960	1.000000	0.000000
Credit_History	-0.018615	0.011134	-0.000607	-0.004705	1.000000

```
y=encoder.fit_transform(y)
```

 y

```
array([[1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1,
        0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0,
        0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1,
        1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1,
        1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0,
        1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1,
        1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1,
        1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1,
        0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0,
        1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0,
        1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1,
        1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0,
        1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1,
        1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0]])
```

In [94]:

```
X=X.values
X
```

Out[94]:

```
array([[ 1.,  0.,  0., ..., 360.,  1.,  2.],
       [ 1.,  1.,  1., ..., 360.,  1.,  0.],
       [ 1.,  1.,  0., ..., 360.,  1.,  2.],
       ...,
       [ 1.,  1.,  1., ..., 360.,  1.,  2.],
       [ 1.,  1.,  2., ..., 360.,  1.,  2.],
       [ 0.,  0.,  0., ..., 360.,  0.,  1.]])
```

In [112]:

```
X_test= X_test.values
X_test
```

Out[112]:

```
array([[ 1.,  1.,  0., ..., 360.,  1.,  2.],
       [ 1.,  1.,  1., ..., 360.,  1.,  2.],
       [ 1.,  1.,  2., ..., 360.,  1.,  2.],
       ...,
       [ 1.,  0.,  0., ..., 360.,  1.,  1.],
       [ 1.,  1.,  0., ..., 360.,  1.,  0.],
       [ 1.,  0.,  0., ..., 180.,  1.,  0.]])
```

In [116]:

```
#Deployment of the model
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# DEcision tree based on entropy,It can be gini also
Dtree=DecisionTreeClassifier(random_state=1,criterion="entropy")
```

In [114]:

```
Dtree.fit(X,y)
```

Out[114]:

```
DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='entropy',
                      max_depth=None, max_features=None, max_leaf_nodes=None,
                      min_impurity_decrease=0.0, min_impurity_split=None,
                      min_samples_leaf=1, min_samples_split=2,
                      min_weight_fraction_leaf=0.0, presort='deprecated',
                      random_state=1, splitter='best')
```

In [115]:

```
y_pred=Dtree.predict(X_test)
```

In [118]:

```
print("Predicted values:")  
print(y_pred)
```

Predicted values:

```
[ 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y'  
 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N'  
 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'Y'  
 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'N' 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'Y'  
 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'Y'  
 'Y' 'N' 'Y' 'Y' 'N' 'N' 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'N'  
 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'Y' 'Y' 'Y' 'N' 'Y'  
 'N' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'Y' 'N' 'Y'  
 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'N' 'N' 'Y' 'Y' 'Y' 'Y' 'N'  
 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'N' 'N' 'Y' 'N' 'Y' 'N' 'N' 'N' 'Y' 'Y' 'Y' 'Y'  
 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'N' 'N' 'N' 'Y' 'Y' 'N' 'Y'  
 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'N' 'Y' 'Y' 'Y'  
 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'N' 'N'  
 'Y' 'N' 'N' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'Y' 'N' 'N' 'Y'  
 'Y' 'Y' 'N' 'N' 'Y' 'Y' 'Y' 'N' 'N' 'Y' 'N' 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'N'  
 'Y' 'Y' 'Y' 'N' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'N' 'Y'  
 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'Y'  
 'N' 'N' 'Y' 'Y' 'N' 'Y' 'Y' 'Y' 'Y' 'Y' 'Y' 'N' 'N' 'Y' 'Y' 'Y' 'N' 'Y'  
 'Y' 'Y' 'N' 'Y' 'Y' 'N' 'Y']
```

In []:

In []: